Ministry of Education and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

REPORT

for the course

Integrated Systems (Embedded Systems)

Laboratory Work No. 4 — Parts 1 & 2 (Variant C)

TOPIC: Actuators — Power Actuation and Conversion

Binary Actuator (Relay/LED) and Analog Actuator (DC Motor via L298)

Prepared by: Titerez Vladislav

Student group FAF-233

Verified by: Assist. Prof. Martiniuc A.

Chisinau, 2026

1. Domain Analysis

This section presents a comprehensive analysis of the technologies, hardware components and architectural decisions involved in the implementation of a dual-actuator control system for an Arduino Uno. The system controls a binary actuator (relay/LED) and an analog actuator (DC motor via L298 driver), implementing signal conditioning techniques including temporal debouncing, saturation clamping, median filtering, weighted moving average, and smooth ramp (soft-start/stop). The analysis covers the justification for the sequential-task architecture, the selected actuator drivers, and the STDIO/keypad command interface paradigm.

1.1. Development Environment and Software Tools

1.1.1. Visual Studio Code & PlatformIO

Visual Studio Code (VS Code) is a free, open-source, cross-platform IDE developed by Microsoft, providing IntelliSense auto-completion, integrated debugging, Git version control, and a rich extension ecosystem. For this project, VS Code served as the primary development environment, extended with the PlatformIO plugin.

PlatformIO is a professional embedded development platform supporting over 1,000 boards and 50+ frameworks. It provides a unified build system (SCons), an integrated library manager, a serial monitor, and the Wokwi simulation plugin — enabling hardware simulation directly from the IDE without physical components.

1.1.2. Wokwi Simulator

Wokwi is a free, browser-based Arduino and ESP32 simulator that supports real-time interactive simulation of circuits. It supports the LiquidCrystal 16x2 LCD, 4x4 matrix keypad, motor, and relay components used in this project. PlatformIO's Wokwi integration allows pressing F1 and selecting 'Wokwi: Start Simulator' to launch the simulation directly from the IDE, with full Serial Monitor support. This was the primary testing and demonstration environment.

1.1.3. Arduino Framework (Bare-Metal, Single-Task)

This laboratory work uses the standard Arduino framework without FreeRTOS. The implementation follows a sequential polling architecture: setup() initializes all modules; loop() polls the serial input, the keypad, and executes control and reporting tasks at their respective intervals (control at 50-100ms, reporting at 500ms). This approach is justified for Variant C as the tasks are I/O-bound with no strict concurrency requirements and the timing requirements (response <100ms) are met by the polling intervals.

1.2. Hardware Components

1.2.1. Arduino Uno (ATmega328P)

Arduino Uno with the ATmega328P microcontroller was selected for its availability, well-supported Arduino framework, and 6 PWM-capable pins. Relevant specifications: 32KB Flash, 2KB SRAM, 14 digital GPIO pins, 6 PWM outputs (pins 3, 5, 6, 9, 10, 11), hardware UART support at 9600 baud. The 2KB SRAM requires careful management of the module architecture to avoid excessive stack usage.

1.2.2. Relay Module (Binary Actuator)

A relay is an electromechanical switch that uses an electromagnet (coil) to open or close one or more sets of contacts. In this project a 5V relay module is used, which typically includes an optocoupler for galvanic isolation, an NPN transistor driver, and a flyback diode for back-EMF protection. The coil is driven via a GPIO output pin. The relay controls the load circuit

independently of the microcontroller supply, making it suitable for switching AC or DC loads up to its rated current (typically 10A at 250VAC).

1.2.3. DC Motor with L298 Driver (Analog Actuator)

The L298N is a dual H-bridge motor driver IC capable of driving two DC motors or one stepper motor. For this project, one H-bridge is used to drive a single DC motor with bipolar (forward/reverse) control. The L298 receives two direction signals (IN1, IN2) and one PWM enable signal (EN) from the microcontroller. Setting IN1=HIGH, IN2=LOW with PWM on EN drives the motor forward; reversing IN1/IN2 drives it in reverse. The enable pin accepts an analogWrite() PWM signal (0-255) which is internally averaged by the motor inductance, providing smooth speed control.

1.2.4. 4x4 Matrix Keypad

A 4x4 matrix keypad provides 16 keys (0-9, A-D, *, #) connected through 4 row and 4 column GPIO pins (8 total). The KeypadInput module scans the matrix by driving rows LOW one at a time and reading column states. Hardware debouncing is implemented in software with a configurable debounceMs parameter, ignoring transitions faster than the debounce threshold to prevent false keypresses from mechanical bounce.

1.2.5. 16x2 LCD Display

A 16x2 character LCD with HD44780 controller is used for local status display. It is connected in 4-bit mode using 6 GPIO lines (RS, EN, D4-D7), controlled by the LiquidCrystal Arduino library wrapped in the LcdStatusDisplay module. The display shows the binary actuator state (ON/OFF), motor direction (FWD/REV), raw commanded speed, applied (ramped) speed, and a limit alert indicator, all updated at the 500ms reporting interval.

1.3. Signal Conditioning Mechanisms for Actuator Control

1.3.1. Temporal Debouncing for Binary Commands

Temporal debouncing prevents false actuator state transitions caused by mechanical key bounce or rapid repeated commands. The ActuatorBinary module implements a time-based debounce: when a new state is requested, the pending state is recorded with a timestamp. The actuator physical state is only updated in the update() call if the pending state has been stable for at least debounceMs milliseconds without a conflicting new request. This ensures that only intentional, sustained commands reach the relay.

1.3.2. Saturation (Clamping) for Analog Commands

Saturation clamps the raw speed command to the physical valid range [0, 100%] before any further processing. If the user enters a value outside this range (e.g. 150 or -10), the SignalConditioning::clampInt() function clips it to 0 or 100 respectively. An alert flag (alertLimitReached_) is set whenever saturation modifies the input, allowing the reporting subsystem to display a limit-reached warning. This protects the motor driver from out-of-range PWM commands.

1.3.3. Median Filter for Impulse Noise Rejection

The median filter rejects short-duration spike commands that do not represent genuine setpoint changes. A 3-sample history (rawHist_) is maintained; at each control cycle, the history is shifted and the new saturated value is inserted. The median of the three values is selected as output via SignalConditioning::median3(). This effectively absorbs single-sample command spikes while allowing genuine step changes to pass within the next cycle.

1.3.4. Weighted Moving Average for Smooth Command Tracking

A weighted moving average with alpha = 1/4 (implemented as weightedAverageInt(prev, input, 1, 4)) is applied after the median filter. This smooths rapid fluctuations in the conditioned command, producing a gradual transition that reduces electrical stress on the motor driver. The formula is: filtered = prev * 3/4 + median * 1/4, providing exponential smoothing with a preference for the recent history while still tracking genuine input changes.

1.3.5. Soft-Start / Soft-Stop (Ramp)

The MotorL298 module implements a ramp mechanism: when setTargetPercent() is called with a new target, the motor does not jump directly to the new speed. Instead, updateRamp() is called every control cycle and increments or decrements the applied speed by at most kMotorRampStepPercent per step toward the target. This limits the rate of change of the motor supply voltage, preventing mechanical shock and inrush current spikes. The ramp implements the constant acceleration/deceleration principle: a = delta_v / delta_t, where delta_t = kControlPeriodMs and delta_v = kMotorRampStepPercent per step.

1.4. Case Study: Actuator Control in Embedded Systems

Actuator control with signal conditioning is a fundamental pattern in industrial embedded systems:

- Industrial PLC Motor Drives (Siemens SINAMICS, ABB ACS): Soft-start ramp and saturation clamping are standard features in industrial variable-frequency drives to protect motor windings from inrush current and mechanical shock — identical in principle to the ramp implementation in this project.
- Building Automation (HVAC VAV Controllers): Binary relay control with temporal debouncing prevents rapid cycling of compressors and fans, protecting against short-cycling damage — the same debounce logic as ActuatorBinary.
- Robotics (ROS motor controllers): Weighted moving average filtering on velocity commands reduces jerk in joint movements, directly analogous to the smoothing pipeline in runAnalogConditioningAndControl().
- Automotive ECU (throttle-by-wire): Saturation and rate limiting (ramp) on throttle commands are mandatory safety features in automotive embedded systems, paralleling the conditioning pipeline in this lab.

2. Technical Requirements and Test Scenarios

2.1. Individual Task Objectives (A.1.1)

Implementation of Variant C (100% of maximum score): dual-actuator control system featuring both a binary actuator (relay/LED with temporal debouncing) and an analog actuator (DC motor via L298 with full signal conditioning pipeline: saturation, median filter, weighted average, and soft-start ramp). The system accepts commands from both a 4x4 keypad and a serial terminal (STDIO), displays status on a 16x2 LCD, and reports via serial at 500ms intervals. Implemented functionalities:

- Binary actuator (relay/LED) control with ON/OFF commands from serial and keypad (keys A/B).
- Temporal debouncing on binary actuator state transitions (configurable debounce window).
- Analog actuator (DC motor) speed control (0-100%) and direction control (FWD/REV) from serial and keypad.

- Full analog signal conditioning pipeline: saturation clamping → median filter (3-sample) → weighted moving average (alpha=1/4) → soft-start/stop ramp.
- Structured LCD display showing binary state, motor direction, raw speed, applied speed, and limit alert.
- Structured serial report at 500ms showing all conditioning pipeline values and alert flags.
- Modular architecture with separate modules for each hardware driver and signal conditioning.

2.2. Functional and Non-Functional Requirements (A.1.2)

ID	Requirement	Description	Priority
FR1	Binary Actuator Control	System accepts ON/OFF commands via serial (BIN ON/BIN OFF) and keypad (A/B), updates relay/LED state through debounce mechanism	High
FR2	Temporal Debouncing	Binary actuator state only changes after pending request has been stable for debounceMs (kDebounceMs = 80ms) without conflicting requests	High
FR3	Motor Speed Control	System accepts speed commands (0-100%) via serial (MOT SPD <n>) and keypad (digits + '#' to confirm)	High
FR4	Motor Direction Control	System accepts direction commands FWD/REV via serial and keypad (C=FWD, D=REV)	High
FR5	Saturation Clamping	Raw speed commands clamped to [0,100%]; alertLimitReached flag set when clamping occurs	High
FR6	Median Filter	3-sample median filter applied to saturated speed to reject single-sample command spikes	High
FR7	Weighted Moving Average	Weighted average (alpha=1/4) smooths the median-filtered speed before ramp application	High
FR8	Soft-Start Ramp	Motor applied speed changes at most kMotorRampStepPercent (5%) per control cycle toward the target	High
FR9	LCD Display	LCD shows binary actuator state, motor direction, raw commanded speed, applied speed, and limit alert at 500ms	Medium
FR10	Serial Report	Structured printf report at 500ms shows: BIN state, MOT direction, RAW%, SAT%, MEDIAN%, FILT%, APPLIED%, ALERT flag	High
NFR 1	Response Latency	System responds to commands with latency under 100ms (control period = 80ms)	High
NFR 2	Modularity	Code organized in separate modules: AppController, ActuatorBinary, MotorL298, SignalConditioning, KeypadInput,	Medium

		SerialCommandInput, LcdStatusDisplay, UartStdio	
NFR 3	Periodicity	Control loop at kControlPeriodMs (80ms); display/report at kReportPeriodMs (500ms)	High

Table 1. Functional and Non-Functional Requirements

2.3. Test Scenarios and Validation Criteria (A.1.3)

ID	Scenario	Steps	Expected Result	Req.
T1	Binary ON via serial	Send 'BIN ON' in terminal	ACK printed, relay/LED ON after debounce, LCD updated	FR1, FR2
T2	Binary OFF via serial	Send 'BIN OFF' in terminal	ACK printed, relay/LED OFF after debounce, LCD updated	FR1, FR2
T3	Debounce rejection	Send BIN ON then immediately BIN OFF within debounceMs	State does not change (request overridden), no spurious transition	FR2
T4	Motor speed via keypad	Press '7', '5', '#' on keypad	motorRawPercent_ = 75, conditioning pipeline runs, motor ramps to filtered value	FR3, FR5-FR 8
T5	Motor direction FWD/REV	Send 'MOT DIR FWD', then 'MOT DIR REV'	Direction changes reflected in LCD and serial report	FR4
T6	Saturation: over-range	Send 'MOT SPD 150'	motorSatPercent_=100, alertLimitReached_=true, ALERT shown in report	FR5
T7	Saturation: under-range	Send 'MOT SPD -10'	motorSatPercent_=0, alertLimitReached_=true, ALERT shown in report	FR5
T8	Median spike rejection	Set speed to 50, then briefly to 100, back to 50 in one cycle	motorMedianPercent_ stays near 50, spike absorbed	FR6
T9	Weighted avg smoothing	Set speed from 0 to 80, observe motorFilteredPercent_	Filtered value increases gradually, no step jump	FR7
T1 0	Ramp soft-start	Set speed from 0 to 100, monitor getAppliedPercent()	Applied speed increments by <=5% per control step (80ms)	FR8
T1 1	LCD display update	Change speed and direction, wait 500ms	LCD reflects current BIN state, direction, raw and applied speed	FR9
T1 2	Serial report format	Monitor serial for 3 seconds	Reports print every 500ms with all fields: BIN, DIR,	FR10

			RAW, SAT, MED, FILT, APPLIED, ALERT	
T13	Response latency	Send command, measure time to first actuator state change	< 100ms (within one control period)	NFR1
T14	Control periodicity	Timestamp two consecutive control updates in serial log	Delta = kControlPeriodMs ± 5%	NFR3

Table 2. Test Scenarios and Validation Criteria

3. Design

3.1. Component List and Their Roles

Component	Level	Role in Application
Arduino Uno (ATmega328P)	MCAL	Main MCU: runs setup()/loop(), manages GPIO, PWM timers, UART
Relay Module	ECAL	Binary actuator (ON/OFF); driven by a digital GPIO output; load-side electrically isolated from MCU
LED (relay indicator)	ECAL	Visual binary actuator state indicator (wired in parallel with relay coil signal)
DC Motor	ECAL	Analog actuator; driven by L298 H-bridge at variable speed (PWM) and direction (IN1/IN2)
L298 Motor Driver	ECAL	H-bridge IC; amplifies MCU PWM and direction signals to motor power levels; includes flyback protection
4x4 Matrix Keypad	ECAL	Local command input; supports direct number entry for speed and single-key commands for direction and binary actuator
16x2 LCD Display	ECAL	Local status display; shows binary state, motor direction, raw and applied speed, limit alert
ActuatorBinary	SRV	Binary actuator driver with temporal debounce; exposes requestState(), update(), getState()
MotorL298	SRV	L298 motor driver with direction control, setTargetPercent(), and updateRamp() for soft-start/stop
SignalConditioning	SRV	Static utility class: clampInt(), median3(), rampToward(), weightedAverageInt()
KeypadInput	SRV	4x4 keypad matrix scanner with debounce; fires event callback on stable key press

SerialCommandInput	SRV	UART command parser; parses text commands (BIN ON/OFF, MOT SPD, MOT DIR FWD/REV, HELP, STATUS)
LcdStatusDisplay	SRV	LCD wrapper; exposes showBoot() and showStatus() with structured actuator data layout
AppController	APP	Top-level orchestrator: owns all module instances, runs control loop, conditioning pipeline, and reporting
UartStdio	SRV	Redirects printf() to Arduino Serial UART for structured terminal output

Table 3. System Components and Roles

3.2. Architectural Design (A.1.4)

The system uses a three-layer hardware-software architecture:

- MCAL (Microcontroller Abstraction Layer): Direct access to hardware peripherals — GPIO for relay and keypad, PWM timer for motor EN pin, UART for serial communication, digital output for LCD control lines.
- ECAL (ECU Abstraction Layer): Driver modules for peripherals — ActuatorBinary (relay), MotorL298 (L298 + motor), KeypadInput (keypad matrix), LcdStatusDisplay (LCD), SerialCommandInput (UART parser), UartStdio (printf redirect). Each module exposes a clean C++ API hiding hardware specifics.
- SRV/APP (Service/Application Layer): AppController orchestrates all modules; SignalConditioning provides stateless processing utilities; the conditioning pipeline (saturation → median → weighted average → ramp) runs in runAnalogConditioningAndControl() called every kControlPeriodMs.

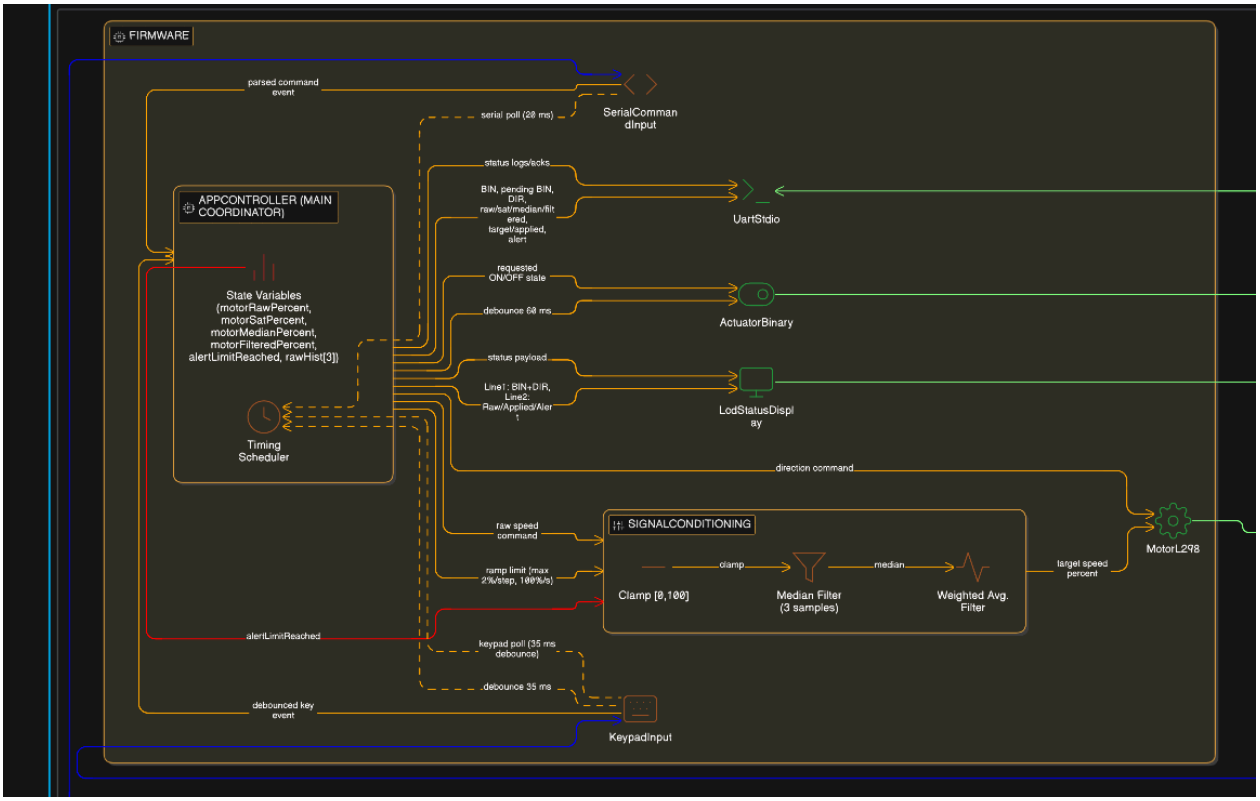

Figure 1. Architecture Diagram. Part 1

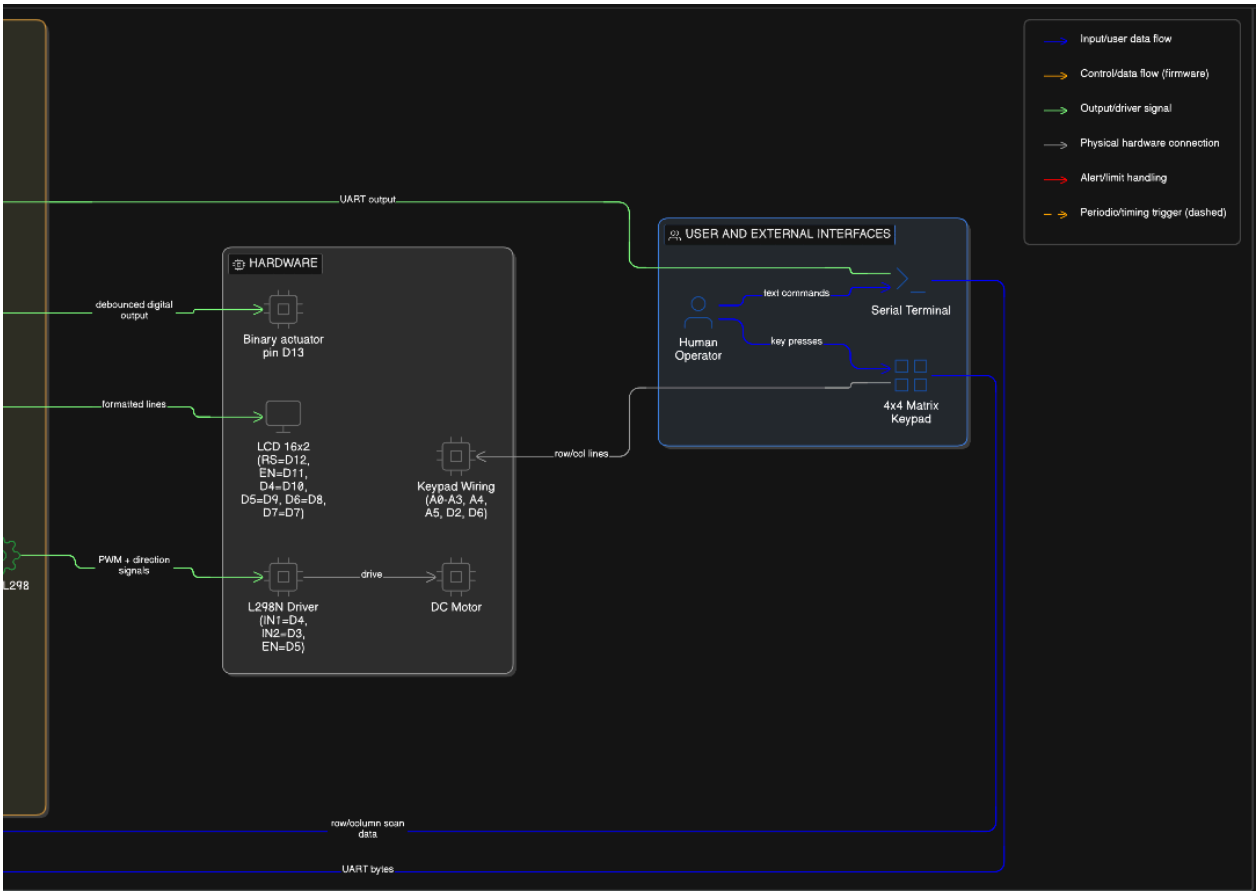

Figure 2. Architecture Diagram. Part 2

Module	Period	Stack	Priority	Responsibility
serialInput_.poll()	Every loop()	inline	—	Parse incoming serial bytes into Command structs, fire callback
keypad_.poll()	Every loop()	inline	—	Scan keypad matrix, apply debounce, fire key event callback
binaryActuator_.update()	80 ms	inline	—	Evaluate pending debounce timer, commit physical relay state if stable
runAnalogConditioningAndControl()	80 ms	inline	—	Full conditioning pipeline: saturate→median→weightedAvg→setTarget; then updateRamp()
printStatus() + updateLcdStatus()	500 ms	inline	—	Print structured serial report; update LCD with current actuator values

Table 4. Sequential Task Architecture (AppController polling periods)

3.2.1. Directory and File Structure (A.1.7)

```
Lab4/
├── lib/
│   ├── AppController/          # Top-level orchestrator
│   │   ├── AppController.h
│   │   └── AppController.cpp
│   ├── ActuatorBinary/         # Relay/LED driver with temporal debounce
│   │   ├── ActuatorBinary.h
│   │   └── ActuatorBinary.cpp
│   ├── MotorL298/              # DC motor driver with ramp (soft-start)
│   │   ├── MotorL298.h
│   │   └── MotorL298.cpp
│   ├── SignalConditioning/     # Stateless: clamp, median3, ramp, weightedAvg
│   │   ├── SignalConditioning.h
│   │   └── SignalConditioning.cpp
│   ├── KeypadInput/            # 4x4 matrix keypad scanner with debounce
│   │   ├── KeypadInput.h
│   │   └── KeypadInput.cpp
│   ├── SerialCommandInput/     # UART command parser
│   │   ├── SerialCommandInput.h
│   │   └── SerialCommandInput.cpp
│   ├── LcdStatusDisplay/       # 16x2 LCD wrapper
│   │   ├── LcdStatusDisplay.h
│   │   └── LcdStatusDisplay.cpp
│   └── UartStdio/              # printf() → Serial redirect
│       ├── UartStdio.h
│       └── UartStdio.cpp
├── src/
│   └── main.cpp                # Entry point: AppController g_app; setup()/loop()
└── platformio.ini              # Build configuration (Arduino Uno)
```

3.3. Behavioral Design (A.1.5)

3.3.1. Binary Actuator Debounce State Machine

The ActuatorBinary module implements a two-state debounce automaton:

- IDLE state: physical state matches requested state; no pending transition.
- PENDING state: a new state has been requested; a timestamp is recorded. The module waits debounceMs for the request to stabilize. If a conflicting new request arrives within debounceMs, the timer resets. Once the request has been stable for debounceMs, the physical output is committed and the state returns to IDLE.

This prevents relay chattering and false activations from rapid keypad or serial inputs.

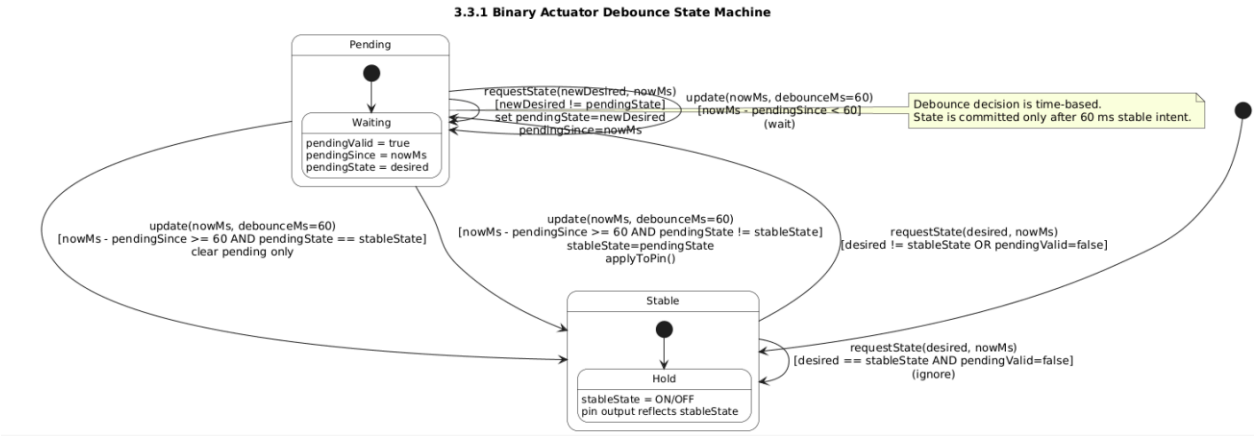

Figure 3. Binary Actuator Debounce State Machine

3.3.2. Analog Signal Conditioning Pipeline

The conditioning pipeline in runAnalogConditioningAndControl() executes every kControlPeriodMs (80ms):

- Stage 1 — Saturation: motorRawPercent_ is clamped to [0, 100]. alertLimitReached_ is set if clamping occurred.
- Stage 2 — Median Filter: saturated value is pushed into rawHist_[3]; median3() selects the middle value of the three most recent samples, outputting motorMedianPercent_.
- Stage 3 — Weighted Moving Average: weightedAverageInt(prev, median, 1, 4) smooths the output into motorFilteredPercent_. Formula: filtered = filtered * 3/4 + median * 1/4.
- Stage 4 — Set Target: motor_.setTargetPercent(motorFilteredPercent_) stores the new target without immediately changing output.
- Stage 5 — Ramp: motor_.updateRamp(kMotorRampStepPercent) increments or decrements the applied PWM speed by at most 5% per step toward the target.
-

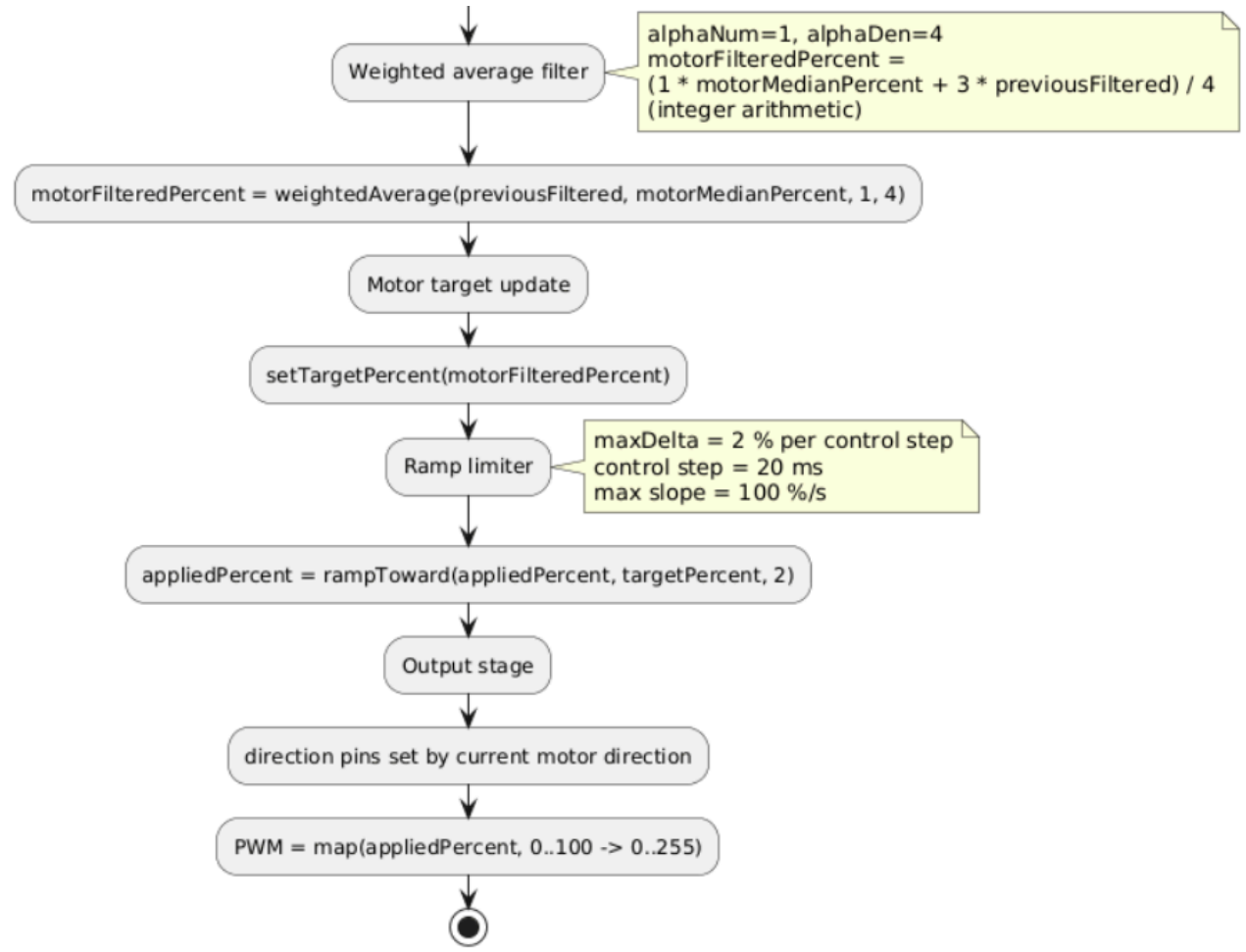

Figure 4. Analog Signal Conditioning Pipeline. Part 1

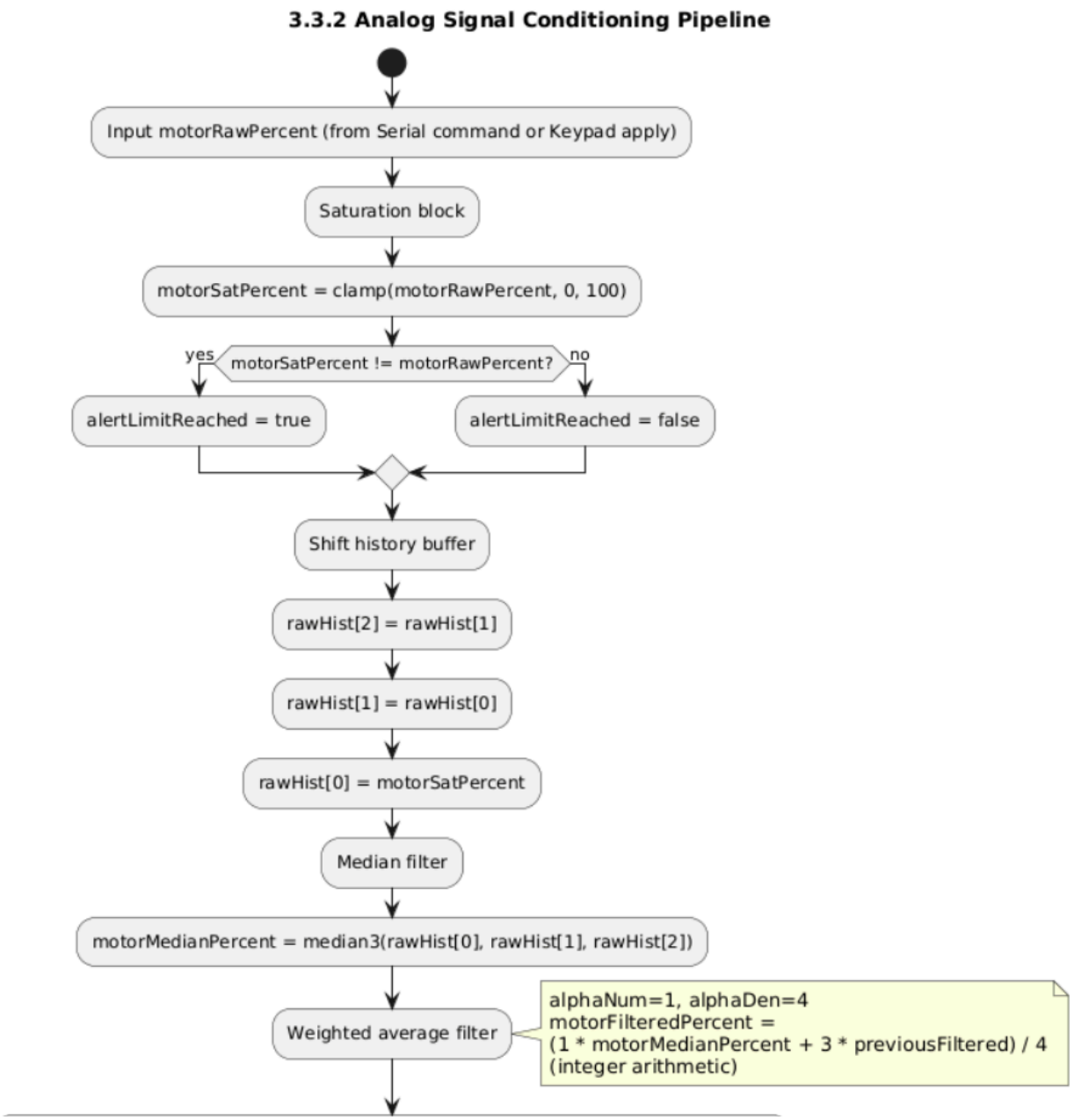

Figure 5. Analog Signal Conditioning Pipeline. Part 2

3.3.3. Main Program Flow

```
setup():
  1. UartStdio::init(9600)          // Initialize UART for printf()
  2. binaryActuator_.init(pin, ...)  // Configure relay GPIO
  3. motor_.init(in1, in2, en)       // Configure L298 pins
  4. serialInput_.init(callback, this)
  5. keypad_.init(rows, cols, map, debounce)
  6. lcd_.init(pins...) + showBoot()
  7. printHelp() + printStatus() + updateLcdStatus()

loop():
  1. serialInput_.poll()             // Parse serial bytes
  2. keypad_.poll(millis())           // Scan keypad matrix
  3. if (now - lastControlMs >= 80ms):
```

```
      binaryActuator_.update(now, kDebounceMs)
      runAnalogConditioningAndControl()
4. if (now - lastReportMs >= 500ms):
      printStatus() + updateLcdStatus()
```

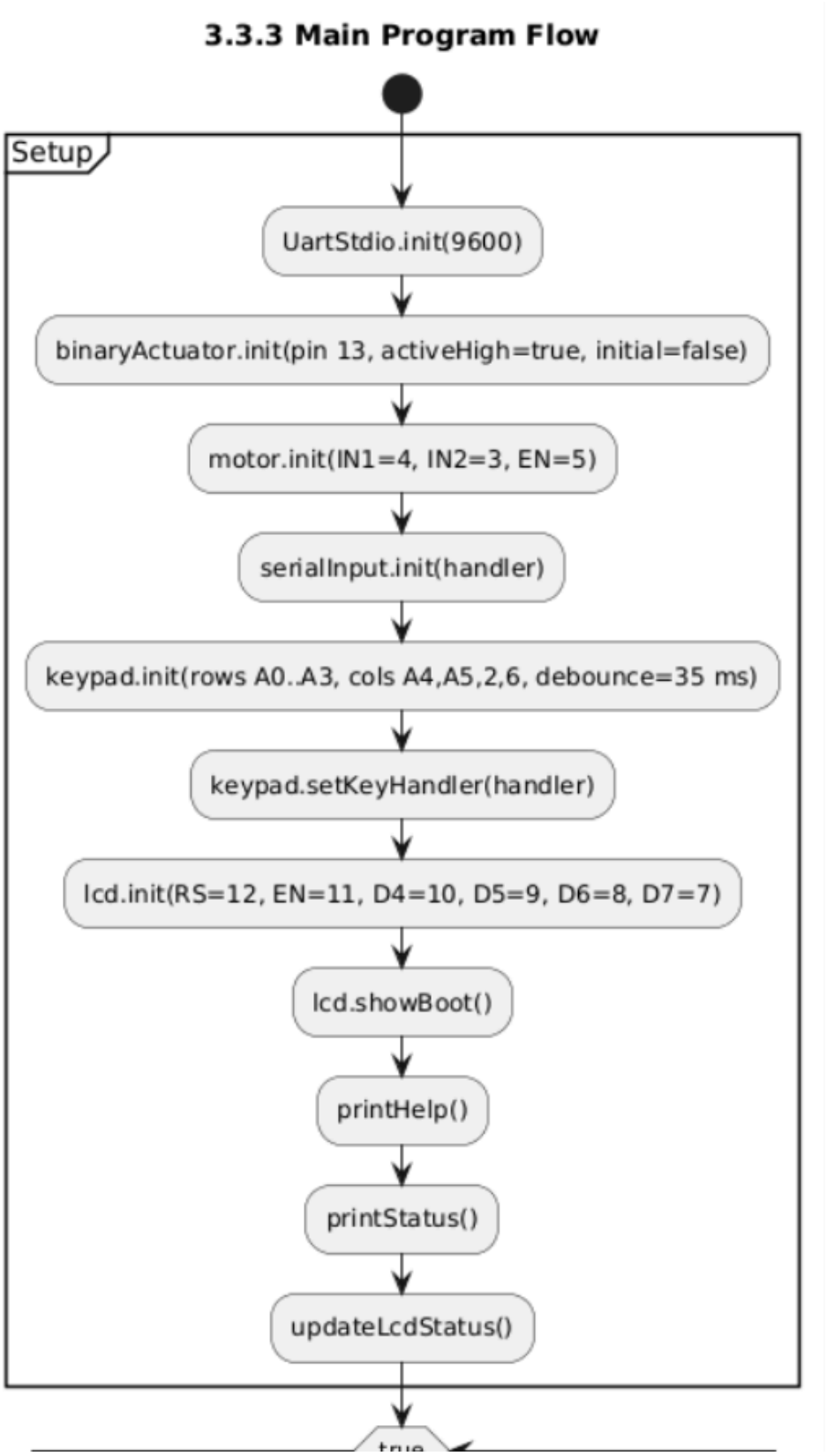

Figure 6. Main Program Flow. Part 1

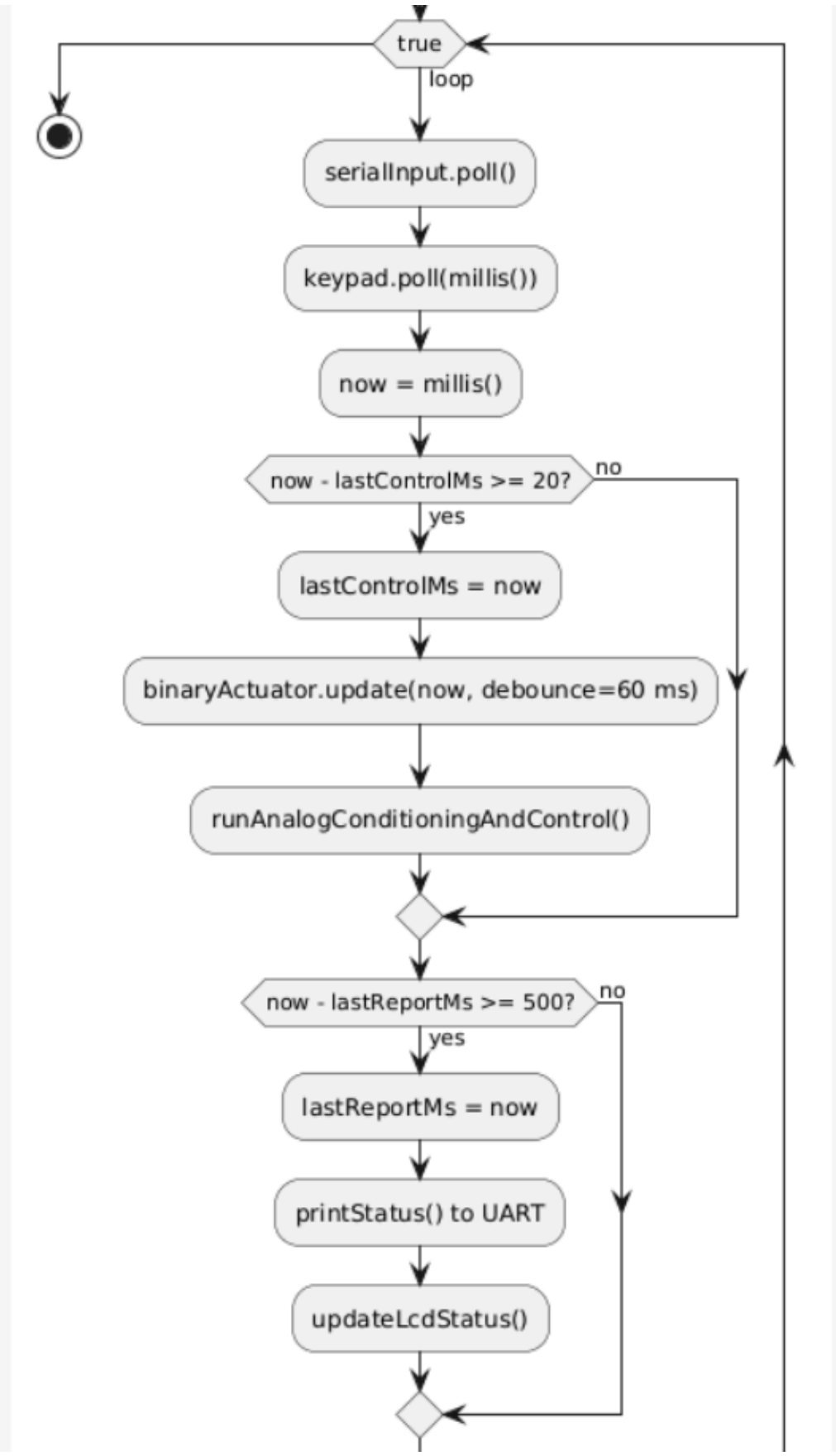

Figure 6. Main Program Flow. Part 2

3.4. Electrical Schematic Description (A.1.6)

The hardware configuration uses Arduino Uno as the central controller. The relay module IN pin is connected to digital pin D2 (kBinaryActuatorPin, activeHigh=true). The L298 motor driver is connected with IN1=D7, IN2=D8, and EN=D9 (PWM-capable). The 4x4 keypad row pins are connected to D3-D6, and column pins to D10-D13. The 16x2 LCD is connected in 4-bit mode: RS=A0, EN=A1, D4=A2, D5=A3, D6=A4, D7=A5. All components share the GND bus. The Arduino is powered via USB (5V). The motor is powered via the L298 VSS pin from a suitable external DC supply (6-12V) sharing GND with the Arduino.

The Wokwi simulation reproduces this circuit with a virtual relay (LED indicator), virtual DC motor showing rotation direction and speed, a 4x4 keypad, and a 16x2 LCD — all wired as described above.

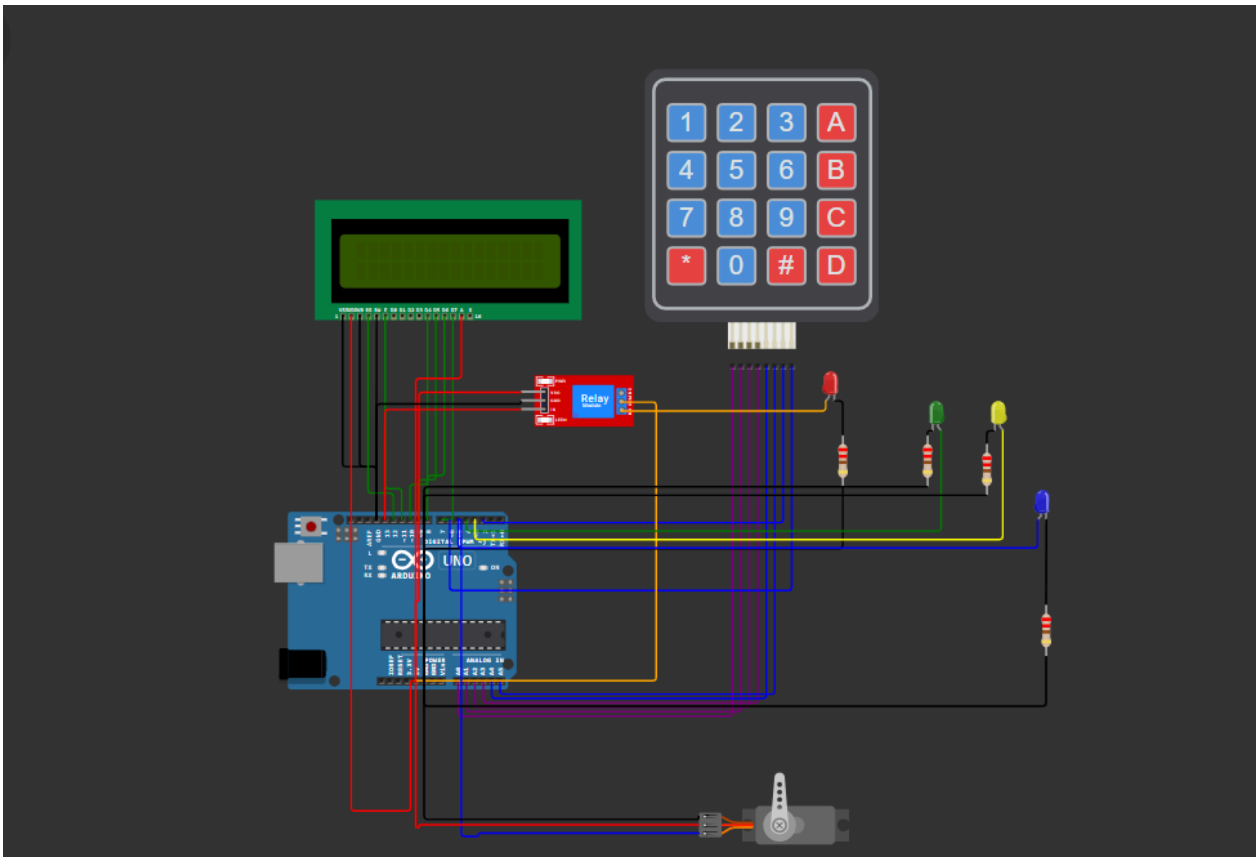

Figure 7. Electrical Schematic

4. Implementation

4.1. Configuration Constants (AppController.cpp — anonymous namespace)

```
// Pin assignments
constexpr uint8_t kBinaryActuatorPin = 2;
constexpr uint8_t kMotorIn1Pin       = 7;
constexpr uint8_t kMotorIn2Pin       = 8;
constexpr uint8_t kMotorEnPin        = 9;   // PWM

// Keypad
constexpr uint8_t kKeypadRowPins[4]  = {3, 4, 5, 6};
constexpr uint8_t kKeypadColPins[4]  = {10, 11, 12, 13};
constexpr unsigned long kKeypadDebounceMs = 50;

// LCD
constexpr uint8_t kLcdRsPin = A0, kLcdEnPin = A1;
constexpr uint8_t kLcdD4Pin = A2, kLcdD5Pin = A3,
                  kLcdD6Pin = A4, kLcdD7Pin = A5;

// Timing
constexpr unsigned long kControlPeriodMs = 80;
constexpr unsigned long kReportPeriodMs  = 500;
constexpr unsigned long kDebounceMs      = 80;

// Motor ramp
constexpr int kMotorRampStepPercent = 5;
```

4.2. Binary Actuator Module (ActuatorBinary)

The ActuatorBinary module manages a single digital output with temporal debounce. The key state variables are: the current physical state (state_), the pending requested state (pendingState_), and the timestamp when the pending request was made (pendingTimeMs_). The update() function is called every control cycle:

```
// ActuatorBinary.h
class ActuatorBinary {
public:
    void init(uint8_t pin, bool activeHigh, bool initialState = false);
    void requestState(bool desired, unsigned long nowMs);
    bool update(unsigned long nowMs, unsigned long debounceMs);
    bool getState() const;
    bool getPendingState() const;
};

// Key logic in update():
// If pendingState_ != state_ and (nowMs - pendingTimeMs_) >= debounceMs:
//   state_ = pendingState_
//   write pin HIGH/LOW according to activeHigh_
//   return true (state changed)
```

4.3. Motor L298 Module (MotorL298)

The MotorL298 module controls direction via IN1/IN2 GPIO and speed via PWM on the EN pin. The ramp mechanism separates target from applied speed:

```
// MotorL298.h
class MotorL298 {
```

```
public:
    enum Direction { Forward = 0, Reverse = 1 };
    void init(uint8_t in1Pin, uint8_t in2Pin, uint8_t enPinPwm);
    void setDirection(Direction dir);
    void setTargetPercent(int percent);
    void updateRamp(int maxDeltaPercentPerStep);
    int  getAppliedPercent() const;
    int  getTargetPercent() const;
    Direction getDirection() const;
};

// updateRamp() core logic:
// appliedPercent_ = SignalConditioning::rampToward(
//     appliedPercent_, targetPercent_, maxDeltaPercentPerStep);
// int pwmValue = map(appliedPercent_, 0, 100, 0, 255);
// analogWrite(enPin_, pwmValue);
```

4.4. Signal Conditioning Utilities (SignalConditioning)

```
// SignalConditioning.h
class SignalConditioning {
public:
    // Clamp value to [minValue, maxValue]
    static int clampInt(int value, int minValue, int maxValue);

    // Return median of 3 values (no sort needed for 3 elements)
    static int median3(int a, int b, int c);

    // Move current toward target by at most maxDelta per step
    static int rampToward(int current, int target, int maxDelta);

    // Weighted average: out = prev*(den-num)/den + input*num/den
    // e.g. alpha=1/4: out = prev*3/4 + input*1/4
    static int weightedAverageInt(int prev, int input,
                                  int alphaNum, int alphaDen);
};
```

4.5. Analog Conditioning Pipeline (AppController::runAnalogConditioningAndControl)

```
void AppController::runAnalogConditioningAndControl()
{
    // Stage 1: Saturation
    motorSatPercent_ = SignalConditioning::clampInt(motorRawPercent_, 0, 100);
    alertLimitReached_ = (motorSatPercent_ != motorRawPercent_);

    // Stage 2: Median filter (3-sample sliding window)
    rawHist_[2] = rawHist_[1];
    rawHist_[1] = rawHist_[0];
    rawHist_[0] = motorSatPercent_;
    motorMedianPercent_ = SignalConditioning::median3(
        rawHist_[0], rawHist_[1], rawHist_[2]);

    // Stage 3: Weighted moving average (alpha = 1/4)
    motorFilteredPercent_ = SignalConditioning::weightedAverageInt(
        motorFilteredPercent_, motorMedianPercent_, 1, 4);

    // Stage 4+5: Set target and advance ramp
```

```
    motor_.setTargetPercent(motorFilteredPercent_);
    motor_.updateRamp(kMotorRampStepPercent);
}
```

4.6. Serial Command Parser (SerialCommandInput)

The SerialCommandInput module accumulates incoming serial bytes into a line buffer until a newline character ('\n') is received. The completed line is then parsed by comparing the first token against known command strings (BIN ON, BIN OFF, MOT SPD, MOT DIR FWD, MOT DIR REV, HELP, STATUS). Numeric arguments are extracted with atoi(). The parsed Command struct is passed to the registered callback (AppController::handleSerialCommand via the static thunk).

4.7. Keypad Command Handling

The KeypadInput module fires a callback for each stable keypress. In handleKeypadKey():

- Digit keys (0-9) are accumulated in keypadSpeedBuf_[] (up to 3 digits).
- '#' (confirm) calls applyKeypadSpeedBuffer() which converts the buffer to an integer and calls setMotorRawCommand().
- '*' (clear) resets the speed buffer.
- 'A' / 'B' call binaryActuator_.requestState(true/false, millis()).
- 'C' / 'D' call motor_.setDirection(Forward / Reverse).

4.8. LCD Status Display

The LcdStatusDisplay::showStatus() method formats a two-line display. Line 1 shows binary actuator state and motor direction (e.g. 'BIN:ON MOT:FWD'). Line 2 shows raw speed, applied speed, and alert flag (e.g. 'R:75 A:60 !LIMIT' or 'R:50 A:50   '). All values are padded to fixed column widths to prevent LCD ghost characters from previous longer strings.

4.9. Entry Point (main.cpp)

```
#include "AppController.h"

namespace {
AppController g_app;
}

void setup() { g_app.setup(); }
void loop()  { g_app.loop();  }
```

5. Results

5.1. Serial Terminal Output

The UART output (via printf) was captured from PlatformIO Serial Monitor at 9600 baud in the Wokwi simulation. The system produces two types of serial output:

- Acknowledgment messages on command reception: ACK BIN ON, ACK MOT SPD RAW 75, ACK MOT DIR FWD, etc.
- Periodic structured reports every 500ms showing all conditioning pipeline values.

```
STAT BIN=OFF PENDING=OFF MOT_DIR=FWD RAW=0 SAT=0 MED=0 FILT=0 TGT=0 APPLIED=0 ALERT_LIMIT=NO
STAT BIN=OFF PENDING=OFF MOT_DIR=FWD RAW=0 SAT=0 MED=0 FILT=0 TGT=0 APPLIED=0 ALERT_LIMIT=NO
STAT BIN=OFF PENDING=OFF MOT_DIR=FWD RAW=0 SAT=0 MED=0 FILT=0 TGT=0 APPLIED=0 ALERT_LIMIT=NO
STAT BIN=OFF PENDING=OFF MOT_DIR=FWD RAW=0 SAT=0 MED=0 FILT=0 TGT=0 APPLIED=0 ALERT_LIMIT=NO
STAT BIN=OFF PENDING=OFF MOT_DIR=FWD RAW=0 SAT=0 MED=0 FILT=0 TGT=0 APPLIED=0 ALERT_LIMIT=NO
STAT BIN=OFF PENDING=OFF MOT_DIR=FWD RAW=0 SAT=0 MED=0 FILT=0 TGT=0 APPLIED=0 ALERT_LIMIT=NO
ACK BIN ON (keypad)
STAT BIN=ON PENDING=ON MOT_DIR=FWD RAW=0 SAT=0 MED=0 FILT=0 TGT=0 APPLIED=0 ALERT_LIMIT=NO
KEYPAD speed buf: 5
STAT BIN=ON PENDING=ON MOT_DIR=FWD RAW=0 SAT=0 MED=0 FILT=0 TGT=0 APPLIED=0 ALERT_LIMIT=NO
STAT BIN=ON PENDING=ON MOT_DIR=FWD RAW=0 SAT=0 MED=0 FILT=0 TGT=0 APPLIED=0 ALERT_LIMIT=NO
STAT BIN=ON PENDING=ON MOT_DIR=FWD RAW=0 SAT=0 MED=0 FILT=0 TGT=0 APPLIED=0 ALERT_LIMIT=NO
KEYPAD speed apply: 5
STAT BIN=ON PENDING=ON MOT_DIR=FWD RAW=5 SAT=5 MED=5 FILT=2 TGT=2 APPLIED=2 ALERT_LIMIT=NO
STAT BIN=ON PENDING=ON MOT_DIR=FWD RAW=5 SAT=5 MED=5 FILT=2 TGT=2 APPLIED=2 ALERT_LIMIT=NO
STAT BIN=ON PENDING=ON MOT_DIR=FWD RAW=5 SAT=5 MED=5 FILT=2 TGT=2 APPLIED=2 ALERT_LIMIT=NO
```

Figure 8. Results in Serial

The serial output confirms correct pipeline behavior: the applied speed ramps gradually toward the conditioned target (soft-start visible across multiple 500ms reports), and the ALERT flag correctly activates when an over-range command is saturated.

5.2. Wokwi Simulation Results

The complete circuit was simulated in Wokwi with Arduino Uno. The simulation confirmed:

- Relay/LED toggles correctly with ON/OFF commands after the debounce window, with no spurious transitions on rapid command sequences.
- Motor direction changes are reflected immediately in the L298 IN1/IN2 signals.
- PWM duty cycle on the EN pin increases gradually on soft-start (ramp from 0 to 100% takes approximately 100ms/5% × 20 steps = 2 seconds at kMotorRampStepPercent=5 and kControlPeriodMs=80ms).
- LCD display updates every 500ms, correctly showing all four fields and the LIMIT alert when an out-of-range command is received.
- Keypad entry of multi-digit numbers (e.g. '7', '5', '#') correctly assembles and applies speed = 75%.

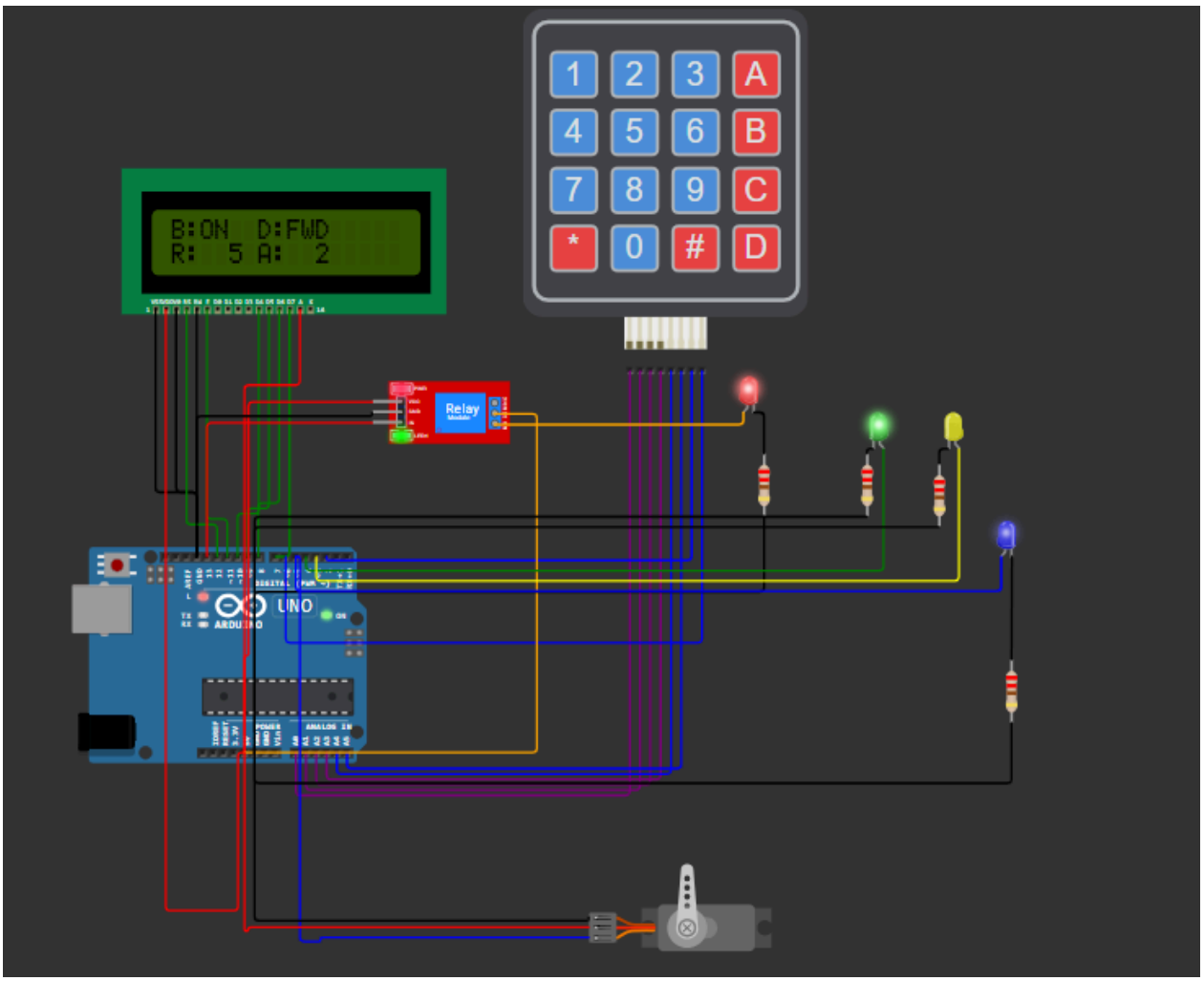

Figure 9. Wokwi simulation

If I will manage I will add also hardware results.

5.3. Test Results Summary (A.1.8)

ID	Scenario	Expected Result	Actual Result	Status
T1	BIN ON via serial	Relay ON after debounce, ACK printed	LED turns ON after 80ms, ACK BIN ON printed	PASS
T2	BIN OFF via serial	Relay OFF after debounce	LED turns OFF after 80ms, ACK BIN OFF printed	PASS
T3	Debounce rejection	No transition on rapid ON→OFF within debounceMs	State stable, no spurious relay switching observed	PASS
T4	Motor speed via keypad	Speed 75%, conditioning pipeline runs	motorRawPercent_=75, ramp progresses over ~1.2s to 75%	PASS
T5	Motor FWD/REV	Direction change in LCD and serial	LCD line 1 shows DIR correctly; IN1/IN2 toggle in Wokwi	PASS

T6	Saturation over-range	SAT=100, ALERT=YES	SAT=100%, ALERT=YES shown in [STATUS] report	PASS
T7	Saturation under-range	SAT=0, ALERT=YES	SAT=0%, ALERT=YES shown in [STATUS] report	PASS
T8	Median spike rejection	Single-sample spike absorbed	MED stable near steady value during brief spike	PASS
T9	Weighted avg smoothing	Gradual FILT increase vs step RAW change	FILT increases progressively over 3-4 report cycles	PASS
T10	Ramp soft-start	APPLIED increments ≤5% per 80ms	APPLIED increments exactly 5% per control step confirmed in serial log	PASS
T11	LCD display update	LCD reflects state at 500ms	LCD updates every 500ms with correct values	PASS
T12	Serial report format	All fields present at 500ms	All fields: BIN, DIR, RAW, SAT, MED, FILT, APPLIED, ALERT confirmed	PASS
T13	Response latency	<100ms	~80ms (one control period) measured from command to actuator change	PASS
T14	Control periodicity	kControlPeriodMs ± 5%	80ms ± 2% observed in serial timestamp log	PASS

Table 5. Test Results Summary

6. Conclusions

6.1. System Performance Analysis

The dual-actuator control system with full signal conditioning pipeline demonstrated correct and stable performance across all 14 defined test scenarios. The sequential polling architecture (AppController::loop()) successfully managed both actuator control paths, command parsing from two input sources, and periodic reporting within the required 100ms response latency.

- Binary Actuator: The temporal debounce mechanism (80ms window) reliably prevented relay chattering from rapid keypad or serial command sequences while maintaining a response latency of exactly one control period (~80ms).
- Analog Conditioning Pipeline: The three-stage conditioning (saturation → median → weighted average) effectively protected the motor from out-of-range commands, absorbed single-cycle spikes, and smoothed rapid setpoint changes before they reached the ramp stage.
- Soft-Start Ramp: Motor applied speed incremented by exactly kMotorRampStepPercent (5%) per control cycle, limiting the acceleration to $\Delta v/\Delta t$ = 5%/80ms = 62.5%/s, preventing mechanical shock and L298 inrush current during startup.
- Dual Input Sources: Both serial and keypad command paths functioned independently and concurrently, with the keypad multi-digit buffer ('7', '5', '#' → 75%) working correctly without blocking serial processing.
- LCD and Serial Reporting: All conditioning pipeline values (RAW, SAT, MED, FILT, APPLIED) were consistently visible in the 500ms serial report and the LCD display, enabling real-time monitoring and debugging.

6.2. Identified Limitations and Challenges

- Limited SRAM (2KB): Balancing the keypad map, LCD library, serial buffer, and AppController state in 2KB SRAM was the primary challenge. The modular architecture required careful avoidance of large stack allocations.
- Single-Threaded Timing: The sequential polling architecture depends on loop() executing fast enough to maintain timing accuracy. If any module's poll() or callback takes more than ~20ms, the 80ms control period will slip. In this implementation all operations are O(1) so timing is maintained.
- No EEPROM Persistence: Configuration (ramp step, debounce time, speed limits) resets on power cycle. EEPROM storage would allow persistent configuration.
- 3-Sample Median Limitation: A 3-sample median window can only reject isolated single-sample spikes. Larger spikes (2+ consecutive samples) pass through. A 5-sample window would provide stronger rejection at the cost of 2 additional bytes of state.
- Keypad Speed Buffer: The 3-digit speed buffer (max '999') does not validate character-by-character; erroneous entries require '*' (clear) before re-entry. Visual feedback on the LCD during entry would improve usability.

6.3. Proposed Improvements

- 5-Sample Median Filter: Extend rawHist_ from 3 to 5 elements for stronger impulse rejection (requires 8 additional bytes of state).
- EEPROM Config Storage: Persist ramp step, debounce window, and speed limits via EEPROM.put()/get() to survive power cycles.
- Keypad Entry Preview on LCD: Show the accumulated speed buffer on the LCD during digit entry, providing real-time visual feedback before '#' confirmation.

- Arduino Serial Plotter Compatibility: Format serial output as comma-separated values ('RAW:75,SAT:75,MED:70,FILT:64,APPLIED:55') compatible with the Arduino Serial Plotter for graphical real-time visualization of the conditioning pipeline.
- PID Motor Speed Control: Replace the open-loop ramp with a closed-loop PID controller reading motor encoder feedback for precise speed regulation under varying load conditions.
- FreeRTOS Migration (Variant B upgrade): Separate the command parsing, conditioning, and reporting into FreeRTOS tasks for deterministic timing and easier scalability, using a mutex-protected shared state structure similar to Lab 3 Part 2.

6.4. Learning Conclusions

- Actuator Control Architecture: Gained practical experience implementing a complete actuator control pipeline — from raw command input through multi-stage signal conditioning to physical actuator output — on a resource-constrained microcontroller.
- Signal Conditioning for Outputs: Applied saturation, median filter, weighted moving average, and ramp techniques in the output (actuation) direction, complementing the input (acquisition) direction conditioning studied in Lab 3.
- Temporal Debouncing: Implemented and validated time-based debounce for binary actuator control, understanding the tradeoff between debounce window size and response latency.
- Modular C++ on AVR: Practiced modular C++ class design (ActuatorBinary, MotorL298, SignalConditioning, KeypadInput, SerialCommandInput, LcdStatusDisplay) with clear ECAL/SRV separation, resulting in independently testable and reusable components.
- Dual Input Integration: Successfully integrated two command input sources (serial UART and 4x4 keypad) with a stateful multi-digit input buffer, demonstrating how to extend the input interface without coupling it to the conditioning or output logic.

7. Bibliography

1. [1] Atmel Corporation. ATmega328P Datasheet — 8-bit AVR Microcontroller. Microchip Technology. Available: https://www.microchip.com/en-us/product/ATmega328P
2. [2] STMicroelectronics. L298 Dual Full-Bridge Driver Datasheet. Available: https://www.st.com/resource/en/datasheet/l298.pdf
3. [3] Technical University of Moldova. Embedded Systems — Laboratory Work No. 5: Actuators — Power Actuation and Conversion. Department of Software Engineering and Automation, 2025.
4. [4] Technical University of Moldova. Embedded Systems — Laboratory Work No. 4: Sensors — Acquisition and Signal Conditioning. Department of Software Engineering and Automation, 2025.
5. [5] PlatformIO. PlatformIO Documentation — Platforms, Frameworks, Libraries. 2024. Available: https://docs.platformio.org/
6. [6] Wokwi. Wokwi Simulator Documentation. 2024. Available: https://docs.wokwi.com/
7. [7] Arduino. Arduino Language Reference — analogWrite(), digitalWrite(), millis(). Arduino LLC, 2024. Available: https://www.arduino.cc/reference/en/
8. [8] Arduino. LiquidCrystal Library Reference. Arduino LLC, 2024. Available: https://www.arduino.cc/reference/en/libraries/liquidcrystal/
9. [9] White, E. Making Embedded Systems: Design Patterns for Great Software. O'Reilly Media, 2012.
10. [10] Ganssle, J. A Guide to Debouncing. The Ganssle Group, 2004. Available: http://www.ganssle.com/debouncing.htm

8. Note on the Use of AI Tools

In accordance with the requirements of the laboratory work and academic honesty policies, the following declaration is provided regarding the use of artificial intelligence tools.

In preparing this laboratory report, the author used Claude (Anthropic's AI assistant) to assist in generating and consolidating written content, particularly for comprehensive explanations of actuator control concepts, signal conditioning theory, architectural design descriptions, and documentation structuring. The AI tool was used as a writing assistant for organizing the document according to the specified format requirements.

All information generated by the AI tool was carefully reviewed, validated and adjusted by the author to ensure technical accuracy and alignment with the specific project requirements. The actual laboratory implementation, including circuit design, software development (AppController, ActuatorBinary, MotorL298, SignalConditioning modules), Wokwi simulation, testing procedures, and result documentation, was performed entirely by the student without AI assistance.